ULC-ICAM • RÉPUBLIQUE DÉMOCRATIQUE DU CONGO

Mon Miam Miam

Choix des technologies et architecture de l’application

Comprendre la solution avant de concevoir les écrans dans Figma

GROUPE 4
Moise Balaga • Lucas Kandende • Michael Cikuru
Assek Arloni • Kabeya Ryan • Bacirhamya Yedidia
Prudent Kakesa

Application : plateforme web regroupant plusieurs restaurants dès la première version.
Contexte : ULC-ICAM, RDC ; montants en francs congolais (CDF / FC), sans conversion.
Version : 1.0 - Document de conception proposé.
Date : 20 octobre 2026.

Statut : proposition technique à examiner avec le groupe et l’encadrant. La tâche Jira **MMM-24** est en revue. Ce document décrit la solution à réaliser ; il ne certifie pas que les installations, le code et le déploiement sont déjà terminés.

1. But du document et repères

Cette partie transforme les besoins analysés en une organisation technique compréhensible : quels logiciels employer, quelle responsabilité donner à chacun et comment relier les écrans, le serveur et les données. Elle prépare les maquettes, la modélisation et le développement.

1.1 Distinguer ce qui est demandé de ce qui est proposé

Repère	Signification
[CDC] Cahier des charges	Exigence ou indication du document source. Quand une technologie est seulement citée en exemple, cela est précisé.
[GROUPE] Précision confirmée	Contexte ULC-ICAM en RDC, monnaie CDF/FC et plusieurs restaurants obligatoires dès la première version.
[PROPOSITION] Choix de réalisation	Solution conseillée pour satisfaire le besoin. Elle doit être comprise et confrontée aux consignes de l'encadrant avant réalisation.

Source principale : CAHIER DE CHARGE PROJET : Mon Miam Miam, V1.0.2 du 16 septembre 2024. Les choix reprennent la proposition technique préparée pour MMM-24. Les exemples de montants du CDC ne sont ni convertis ni recalculés.

1.2 Sommaire

Partie	Page
1. But du document et repères	2
2. Technologies et correspondance avec le CDC	3
3. Fonctionnement de l'architecture	4
4. Plusieurs restaurants dans une même application	5
5. Outils de travail et contributions individuelles	6
6. Connexion, sécurité et tests	7
7. Hébergement et préparation des postes	8
8. Répartition, décisions et passage à Figma	9
9. Références et vocabulaire essentiel	10

À pouvoir expliquer au professeur : React affiche les écrans, Laravel applique les règles et les droits, PostgreSQL conserve les données. Jira montre les tâches ; Git et GitHub rendent les contributions et les versions vérifiables.

2. Technologies et cahier des charges

Élément	Rôle dans Mon Miam Miam	Origine du choix
React	Construire les écrans : restaurants, menus, panier, commandes et espaces de gestion.	[CDC] React, Vue.js et Angular sont des exemples ; React est le choix proposé.
TypeScript et Vite	TypeScript aide à détecter des incohérences dans le code ; Vite prépare et construit l'interface.	[PROPOSITION] Outils complémentaires, non imposés par le CDC.
PHP / Laravel	Traiter les requêtes, contrôler les droits, calculer les montants et fournir l'API.	[CDC] PHP et Laravel sont cités pour le backend. Laravel 13 est proposé.
PostgreSQL	Conserver les comptes, restaurants, articles, commandes et relations entre ces données.	[CDC] Une base est nécessaire ; son moteur n'est pas imposé. PostgreSQL est proposé.
Laravel Sanctum	Gérer l'identification par session et cookies pour l'application web.	[PROPOSITION] Outil de connexion ; les droits métier doivent être ajoutés séparément.
Laravel et Playwright	Tester les règles, les accès, les API et les parcours dans le navigateur.	[CDC] Tests unitaires, fonctionnels et de bout en bout ; outils particuliers proposés.
GitHub Actions	Exécuter automatiquement les contrôles ; déployer lorsque le processus sera configuré.	[CDC] Étape CI/CD ; GitHub Actions cité en exemple et proposé ici.

2.1 Pourquoi ces choix ?

Le groupe a déjà étudié ces technologies. React permet de développer des écrans interactifs ; Laravel structure les règles et les accès ; une base relationnelle relie les restaurants à leurs menus et commandes. Cette séparation donne aux binômes des responsabilités claires, avec un contrat d'API commun.

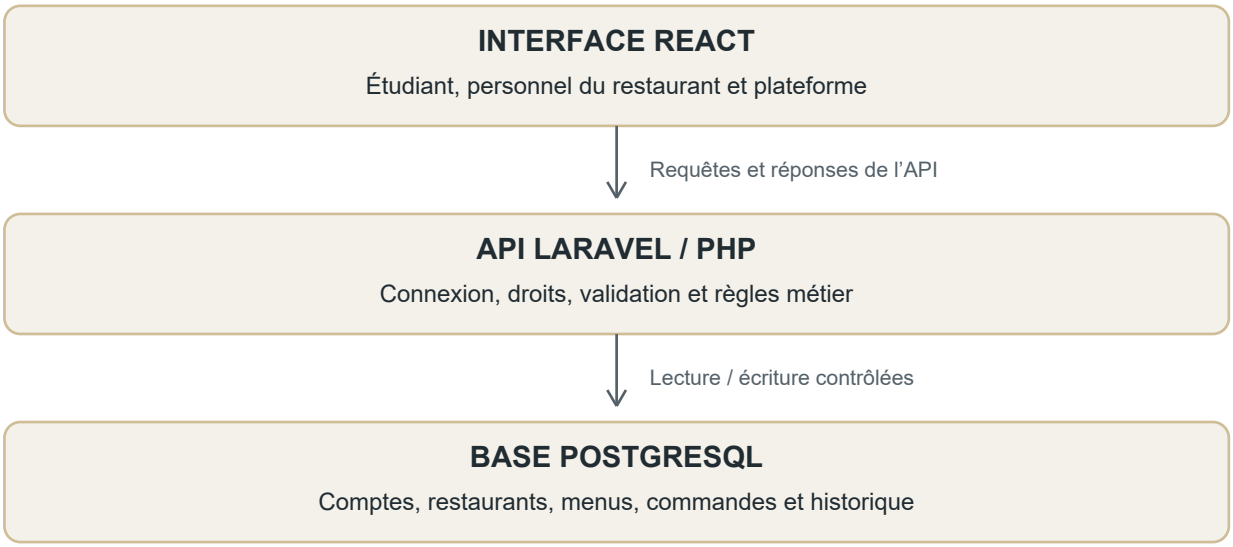
2.2 Autres contraintes à conserver

[CDC] Architecture MVC ou similaire, API documentée, tableaux actualisés en AJAX, Jira, GitHub, maquettes Figma, démonstration frontend sur Vercel et hébergement du backend. La charte utilise **#cfbd97** et **#000000**. Les noms présents dans les ressources du CDC ne sont pas tous des logiciels obligatoires à installer.

Versions : Laravel 13 exige PHP 8.3 au minimum [R1]. La version exacte de PHP et une version PostgreSQL encore supportée devront être fixées avant l'installation. Un choix de technologie ne vaut pas installation réussie.

3. Fonctionnement de l’architecture

[PROPOSITION] Une interface web, un serveur Laravel central organisé en modules et une base partagée suffisent pour plusieurs restaurants. Chaque restaurant dispose de ses données et de ses droits au sein du même système.



3.1 MVC, API et AJAX en termes simples

MVC ou similaire [CDC] : les modèles représentent les données ; les contrôleurs reçoivent les actions ; les règles métier organisent les traitements. React assure les vues au moyen d’une API Laravel. Cette organisation adapte la séparation MVC à une interface indépendante.

API REST : contrat d’échange entre les écrans et le serveur. Il précise les informations demandées, les réponses et les erreurs. **AJAX :** échanges asynchrones permettant de mettre un tableau à jour sans recharger toute la page. La fréquence de rafraîchissement reste à définir ; AJAX ne garantit pas à lui seul une mise à jour instantanée.

3.2 Exemple : passer une commande

1. L’étudiant choisit un restaurant et consulte son menu.
2. React transmet les articles, quantités et le mode de service à Laravel.
3. Laravel contrôle le compte, le restaurant, les disponibilités et les règles ; il recalcule le prix.
4. Le serveur enregistre la commande et ses lignes de façon cohérente dans PostgreSQL.
5. L’étudiant reçoit la confirmation ; le personnel autorisé voit la commande dans son espace.

Principe : le navigateur affiche et transmet les choix ; le serveur prend les décisions qui engagent les prix, les données et les autorisations. Un paiement externe ne s’ajoutera au parcours que si son périmètre est validé.

4. Plusieurs restaurants, données séparées

[GROUPE] Plusieurs restaurants doivent être connectés à l'application dès la V1. La conception doit donc prévoir cette organisation avant les maquettes et le code.

4.1 Qui peut accéder à quoi ?

Acteur	Accès proposé
Étudiant	Choisir différents restaurants et consulter ses propres commandes. Le changement de restaurant n'autorise pas la lecture des commandes d'autres étudiants.
Employé, gérant et administrateur local	Agir dans les restaurants auxquels ils sont explicitement rattachés, selon leurs droits. Les différences entre ces rôles doivent être précisées.
Administrateur de plateforme	Gérer les restaurants et les responsables selon une matrice de droits globale. Ce rôle distinct est une proposition liée au besoin de plusieurs restaurants.

4.2 Première organisation des données [PROPOSITION]

Ensemble	Relations et contrôles essentiels
Utilisateurs et rattachements	Un rattachement relie un utilisateur, un restaurant, un rôle et un statut.
Restaurants et menus	Chaque article appartient à un restaurant ; horaires et disponibilités sont associés au bon établissement.
Commandes et lignes	Chaque commande appartient à un étudiant et à un restaurant. Ses lignes conservent quantités et prix appliqués.
Réclamations et statistiques	La commande définit le restaurant concerné ; les accès et les résultats sont filtrés en conséquence.
Points, parrainages et paiements	La portée locale ou globale des récompenses reste à valider. Le paiement conserve un état distinct de la commande.

La base est partagée, mais les requêtes et les références entre données doivent empêcher les mélanges. Ajouter un champ **restaurant_id** ne suffit pas : le serveur vérifie les droits à chaque action et les relations doivent rester cohérentes.

Panier proposé : un panier et une commande pour un seul restaurant. Si le panier est rempli, changer de restaurant nécessite une confirmation ; le serveur refuse les articles mélangés. Ce fonctionnement doit être validé.
Monnaie : montants en CDF/FC, précision décimale et arrondis documentés, prix recalculés côté serveur, sans conversion des montants du CDC.

5. Quand employer chaque outil ?

Moment	Outil	Travail et preuve à présenter
Analyse et organisation, dès maintenant	Jira	Décrire les tâches, critères de réussite, responsables et état réel. Lier les documents et preuves à la tâche concernée.
Conception des interfaces, prochaine étape	Figma	Dessiner les écrans, leurs états et les parcours. Présenter le choix du restaurant puis le menu, le panier et les espaces de gestion.
Préparation, puis toute modification de fichiers	Git	Enregistrer localement des versions explicites du code et de la documentation, avec l'identité de chaque contributeur.
Dès la création des dépôts, puis en continu	GitHub	Partager les versions, relire les modifications et conserver les contributions personnelles et les demandes d'intégration.
Première version exécutable, puis à chaque évolution	Tests et GitHub Actions	Automatiser les vérifications. Montrer les résultats des tests et corriger les échecs avant intégration.
Démonstration et livraison	Vercel et hébergement backend / base	Partager l'interface et, lorsque tout est relié, une application complète. Fournir les guides et accès adaptés.

5.1 Circuit de contribution proposé

1. Choisir une tâche Jira avec ses critères d'acceptation.
2. Créer une branche liée à sa vraie clé Jira.
3. Réaliser et tester le changement ; enregistrer des commits compréhensibles.
4. Publier sur GitHub et ouvrir une demande d'intégration (pull request).
5. Faire relire et vérifier les tests ; intégrer après corrections.
6. Joindre la preuve à Jira et mettre à jour l'état de la tâche.

5.2 Un dépôt pour chaque personne

[PROPOSITION] Un dépôt principal du groupe et un fork personnel par membre permettent une collaboration structurée. Un fork est une copie GitHub liée au dépôt d'origine. Il faut confirmer que cela satisfait la consigne du professeur si celui-ci exige sept dépôts indépendants.

Identité : l'adresse GitHub peut différer de l'adresse Jira. Chaque membre utilise son propre compte et une identité Git correctement configurée pour que son travail soit attribué.
Pour le professeur : un ticket Jira trace la tâche ; les commits, relectures, tests et démonstrations montrent ce qui a réellement été réalisé.

6. Connexion, sécurité et vérifications

6.1 Identifier une personne et autoriser une action

[PROPOSITION] Sanctum gère la connexion web par session et cookies. Être connecté prouve l'identité du compte ; cela ne donne pas automatiquement accès à toutes les commandes ou à tous les restaurants. Laravel doit contrôler le rôle, le rattachement au restaurant et la ressource demandée [R2-R3].

Exemple : un employé du restaurant A reste interdit d'accès à la commande du restaurant B, même s'il modifie un identifiant dans l'adresse ou la requête. Masquer un bouton dans React ne remplace pas ce contrôle côté serveur.

6.2 Contrôles de conception proposés

Protéger les mots de passe par hachage ; utiliser HTTPS en déploiement ; valider les données et les transitions de commande ; recalculer les montants ; conserver les prix commandés. Les opérations sur commandes et points doivent éviter les traitements répétés et garder un historique utile.

Les secrets, mots de passe et fichiers de configuration contenant des clés ne sont pas publiés sur GitHub. Les exemples de configuration doivent rester sans secrets. Les états de commande et de paiement sont suivis séparément.

6.3 Tests attendus et preuves

Type demandé [CDC]	Exemples utiles pour Mon Miam Miam
Unitaires	Calcul du total, arrondis, remise et règles d'attribution des points lorsque leurs barèmes seront validés.
Fonctionnels / API	Création de commande, accès aux données, rôle par restaurant, refus de panier mixte, validation des informations manquantes.
De bout en bout	Parcours dans le navigateur : choisir un restaurant, commander et retrouver la commande ; traitement par le bon personnel.

Recette minimale sur deux restaurants [PROPOSITION] :
• Créer A et B avec des menus et personnels distincts.
• Passer une commande dans chacun et vérifier les bons prix et destinataires.
• Tenter un accès direct de A vers les données de B : il doit être refusé.
• Refuser une commande mélangeant A et B.
• Désactiver un restaurant : bloquer les nouvelles commandes tout en conservant l'historique.
Ces tests sont à réaliser ; leur réussite n'est pas déclarée à ce stade.

7. Hébergement et préparation des postes

7.1 Publier une interface et une application complète

[CDC] Vercel sert à la démonstration du frontend ; le serveur doit aussi être déployé. **[PROPOSITION]** Construire React avec Vite, publier l'interface sur Vercel et prévoir un environnement PHP compatible pour Laravel ainsi qu'une base PostgreSQL persistante. Le choix d'un fournisseur backend et de ses coûts reste ouvert [R6].

Figma produit les maquettes et le prototype de navigation. L'interface réellement développée peut ensuite être publiée sur Vercel. Un prototype ou une démonstration utilisant de fausses données doit être présenté comme tel ; la recette complète vérifie aussi Laravel et la base.

7.2 Prévoir la connexion dès le choix d'hébergement

Pour la version intégrée, une origine web commune pour l'interface et l'API simplifie la connexion. Autre possibilité : des sous-domaines d'un même domaine principal, avec cookies et règles CORS configurés. Le mode SPA à cookies de Sanctum exige ce domaine principal commun [R3].

> Une interface sur **vercel.app** et une API sur le domaine indépendant d'un autre hébergeur ne doivent pas être supposées compatibles avec ce mode de connexion sans adaptation. Ce point est à résoudre avant le déploiement intégré.

7.3 État observé du poste de Michael

Élément	Observation du 20 octobre 2026	Suite proposée
PHP	Version active 8.0.30 dans XAMPP.	Préparer PHP compatible : Laravel 13 exige au moins 8.3 [R1].
Composer / Node.js / Git	Composer 2.10.3 ; Node.js 24.19.0 ; Git disponible.	Fixer des versions communes et vérifier le démarrage du projet.
Base de données	Client MariaDB 10.4.32 repéré dans XAMPP.	Cela ne prouve pas qu'un serveur fonctionne. Préparer PostgreSQL et tester sa connexion.

[PROPOSITION] Préparer un environnement propre au projet pour préserver les exercices XAMPP existants. Documenter les versions et instructions communes. Structure du dépôt : **frontend/**, **backend/**, **docs/** et **.github/workflows/**.

Statut : les observations ci-dessus sont un inventaire ; cette étape n'a pas installé ou remplacé les logiciels, créé l'application ni validé son fonctionnement.

8. Organisation et passage à Figma

8.1 Répartition proposée à cette étape

Binôme	Responsabilité immédiate	Preuves / tâches Jira
A - Moise Balaga Lucas Kandende	Règles, matrice des droits, documentation et coordination des maquettes.	Décisions et lien Figma ; MMM-8, MMM-9, MMM-45.
B - Michael Cikuru Assek Arloni	Parcours de sélection du restaurant et interfaces de chaque rôle.	Écrans et parcours ; MMM-46, MMM-47, MMM-53.
C - Kabeya Ryan Bacirhamya Yedidia	Données, rattachements aux restaurants et contrat d'API.	Modèle relationnel et API ; MMM-25, MMM-48, MMM-49.
D - Prudent Kakesa Moise Balaga	Préparation des postes, conventions Git, intégration et plan de tests.	Guide et recette sur deux restaurants ; MMM-39, MMM-40, MMM-55.

Le groupe comporte sept membres : Moise intervient dans deux binômes. Planifier son travail de cadrage en premier, puis son appui à l'intégration et aux tests, pour limiter le chevauchement des charges. Cette répartition est une organisation proposée.

8.2 Décisions encore à valider

• Un panier par restaurant et le comportement lors du changement de restaurant.
• Les droits précis de chaque rôle et le cumul éventuel de rattachements.
• La portée de la fidélité et du parrainage, leurs barèmes et déclencheurs.
• Le caractère obligatoire ou bonus du paiement intégré, ambigu dans le CDC.
• L'hébergement, le domaine de connexion, les versions exactes et la conformité des forks à la consigne des dépôts individuels.

8.3 Ce que la prochaine étape Figma doit produire

1. Un plan des écrans et un prototype : connexion, liste des restaurants, menu, panier, confirmation, historique et suivi.
2. Les espaces employé, gérant, administrateur de restaurant et plateforme selon les droits validés.
3. Les états utiles : chargement, liste vide, indisponibilité, erreur, accès refusé et changement de restaurant.
4. Des vues adaptées au téléphone, avec la charte du CDC et les prix en FC.

Suite : examiner cette proposition (MMM-24), consigner les décisions, puis concevoir les maquettes Figma en parallèle du modèle de données et du contrat d'API. Joindre les preuves à Jira. Les dépôts et conventions Git seront préparés avant les premiers fichiers de développement partagés.

9. Références et vocabulaire

Le CDC reste la référence des besoins. Les documentations ci-dessous expliquent les outils proposés et leurs contraintes ; elles ne constituent pas de nouvelles exigences fonctionnelles.

9.1 Sources du projet

[S1] CAHIER DE CHARGE PROJET : Mon Miam Miam, V1.0.2, 16/09/2024 : fonctionnalités, architecture, technologies citées, outils, tests et livrables.
[S2] Précisions du Groupe 4 : ULC-ICAM, RDC, CDF/FC sans conversion et plusieurs restaurants dès la V1.
[S3] Proposition de choix techniques liée à Jira MMM-24, en revue à la préparation de ce document.

9.2 Documentations officielles

[R1] Laravel : versions et prérequis PHP

[R2] Laravel : authentification

[R3] Sanctum : sessions, cookies et application SPA

[R4] Laravel : bases de données prises en charge

[R5] React : démarrer une application avec un outil comme Vite

[R6] Vercel : déployer une interface Vite

[R7] PostgreSQL : versions maintenues

Références vérifiées lors de la préparation des choix, le 20 octobre 2026. Les versions et offres d'hébergement seront revérifiées au moment de l'installation et du déploiement.

9.3 Vocabulaire pour la présentation

Terme	Explication
Frontend / backend	Partie affichée dans le navigateur / partie serveur qui traite les actions et protège les données.
API	Contrat permettant aux écrans et au serveur d'échanger des données.
Commit / branche	Version enregistrée par Git / espace de travail pour préparer une évolution.
Fork / pull request	Copie liée d'un dépôt GitHub / demande de relecture et d'intégration d'un changement.
CI/CD	Vérifications automatiques, puis livraison ou déploiement selon le processus configuré.
Authentification / autorisation	Vérifier qui est connecté / vérifier ce que cette personne peut faire.